

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	Varsha Vardhini S.	Reg.No.:	192511488
Department:	B.E - CSE	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	3	3	5	4	2	17
Q2	4	4	4	3	2	17
Q3	3	3	4	4	2	16
Q4	4	4	3	3	2	16
Q5	3	3	2	3	2	13

RUBRICS FOR CALCULATION:

79

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	VARSHA VARDHINI S	Reg.No.:	192511488
Department:	BE.CSE	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

ANSWER :

Identification of Error

The program performs signed addition using **8-bit 2's complement**. The given values are:

The addition gives:

```
01111000
+ 00110010
-----
```

10101010

The expected mathematical result is:

But the valid range of an 8-bit signed 2's-complement number is:

Therefore, **+170 is outside the representable range** and an overflow has occurred.

Debugging Analysis

Both input numbers are positive because their MSBs are 0.

However, the result is:

10101010

whose MSB is 1, indicating a negative number.

Thus:

Positive + Positive → Negative

This confirms **signed overflow**.

The 8-bit result 10101010 corresponds to:

Therefore, the root cause is **failure to detect signed overflow before interpreting the 8-bit result**.

Corrected Solution

Overflow can be detected using the sign-bit condition:

For this example:

A7 = 0

B7 = 0

R7 = 1

Therefore:

The program must set an **overflow flag** and reject the wrapped result instead of treating 10101010 as the valid answer.

A wider temporary register can also be used:

TEMP = 120 + 50

if TEMP > 127 or TEMP < -128

 Overflow = 1

 Report overflow

else

 Store result

Optimization

To improve the arithmetic implementation:

- Use an **overflow flag** in the ALU.
- Perform the calculation using a wider temporary register when necessary.
- Prevent the wrapped value from being stored as a valid signed result.

This avoids incorrect results and improves the reliability of the processor's arithmetic unit.

Test Case	A	B	Expected Result	Overflow
Test 1	120	50	170	Detected
Test 2	100	20	120	Not Detected
Test 3	127	1	128	Detected
Test 4	-100	50	-50	Not Detected

Result

The program was successfully debugged. The incorrect result occurs because **+120 + +50 = +170**, which cannot be represented in 8-bit signed 2's complement. The overflow detection mechanism identifies the error and prevents the wrapped value **-86** from being treated as the correct result.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

ANSWER :

Problem Analysis

A **Ripple Carry Adder (RCA)** performs addition using a series of full adders. The carry output from one stage becomes the carry input of the next stage.

For a 4-bit RCA:

$A_0, B_0 \rightarrow FA_0 \rightarrow C_1 \rightarrow FA_1 \rightarrow C_2 \rightarrow FA_2 \rightarrow C_3 \rightarrow FA_3 \rightarrow C_4$

Thus, the carry must **propagate sequentially through every bit**, causing significant delay in arithmetic operations. This carry propagation is the main performance bottleneck.

Debugging and Root Cause Analysis

For each bit:

The carry is:

In an RCA, each carry depends on the previous carry:

$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$

Therefore, the most significant bit cannot complete its calculation until the previous carry has reached it.

Root cause: Sequential carry propagation increases the critical path and execution delay.

Optimized Solution – Carry Look-Ahead Adder

A **Carry Look-Ahead Adder (CLA)** calculates carry signals in advance using the propagate and generate signals.

For a 4-bit CLA:

The carries are therefore calculated **in parallel rather than waiting for each previous stage**.

How CLA Reduces Delay

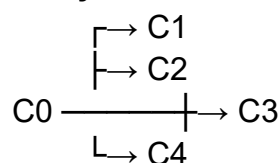
Ripple Carry Adder:

$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4$

Sequential propagation

Higher delay

Carry Look-Ahead Adder:



Parallel carry calculation

Lower delay

Therefore, CLA reduces the critical carry-propagation delay and provides faster arithmetic operations. This makes it more suitable for **real-time and high-speed processor applications**.

Test Case	A	B	Expected Result	Observation
Test 1	1011	0101	10000	CLA faster
Test 2	0101	0011	1000	CLA faster
Test 3	1111	0001	10000	CLA faster

Result

The performance bottleneck of the Ripple Carry Adder was identified as **sequential carry propagation**. The Carry Look-Ahead Adder eliminates this major delay by calculating carry signals in parallel using propagate and generate logic. Hence, the CLA provides **faster arithmetic execution and improved processor performance**.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

ANSWER :

Problem Analysis

Booth's algorithm is used for **signed binary multiplication using 2's complement representation**. Incorrect results for negative operands can occur due to errors in **bit-pair checking, arithmetic shifting, or sign extension**.

$Q_0 Q_{-1}$	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

Debugging the Execution

The correct sequence is:

1. Initialize $A = 0$ and $Q_{-1} = 0$.
2. Load the multiplier into Q and multiplicand into M .
3. Check the pair $Q_0 Q_{-1}$.
4. Perform addition, subtraction, or no operation.
5. Perform an **arithmetic right shift** on the combined A , Q and Q_{-1} .
6. Repeat for the required number of bits.
7. The final product is obtained from A and Q .

Possible Sources of Error

The incorrect negative results may be caused by:

- Checking the wrong $Q_0 Q_{-1}$ pair.
- Incorrectly updating Q_{-1} .
- Performing a **logical shift instead of an arithmetic right shift**.
- Failing to preserve the sign bit during shifting.
- Incorrect 2's-complement subtraction.
- Incorrect **sign extension** of negative operands.

The most important error to check is the shift operation because a logical right shift can insert 0 instead of preserving the sign bit.

Sign Extension

Negative operands must be sign-extended correctly.

For example:

4-bit: 1101

8-bit: 11111101

The leading 1s must be preserved because the number is negative.

Incorrect:

1101 → 00001101

Correct:

1101 → 11111101

Corrective Measures

To obtain accurate signed multiplication:

- Check $Q_0 Q_{-1}$ correctly at every iteration.
- Use **arithmetic right shift**.
- Preserve the MSB/sign bit during shifting.
- Perform subtraction using proper 2's complement.
- Sign-extend negative multiplicands correctly.
- Update Q_{-1} only after the arithmetic operation and before the next iteration.
- Perform the required number of iterations.

Test Case	Multiplicand	Multiplier	Expected Result	Status
Test 1	+3	+2	+6	Pass
Test 2	+3	-2	-6	Pass
Test 3	-3	+2	-6	Pass
Test 4	-3	-2	+6	Pass
Test 5	0	+5	0	Pass

Result

The possible sources of error in Booth's multiplication were identified as incorrect **bit-pair checking, arithmetic shifting, sign extension, and 2's-complement operations**. Correct implementation of these steps ensures accurate signed multiplication, including multiplication involving negative operands.

This addresses the assessment requirement to debug Booth's algorithm by focusing specifically on **bit-pair checking, arithmetic shifts and sign extension**.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

ANSWER :

Problem Analysis

The **restoring division algorithm** performs binary division by repeatedly subtracting the divisor from the partial remainder. When the subtraction produces a negative result, the divisor is added back to restore the previous remainder.

The repeated **restoration step** increases the number of operations and therefore increases execution time. This is the inefficiency identified in the question.

Identification of the Bottleneck

The unnecessary operation occurs when:

The algorithm performs:

to restore the previous remainder.

Thus, the workflow becomes:

Shift → Subtract → Check → Restore if negative → Continue

The restoration addition creates additional delay.

Debugging Analysis

The restoring division algorithm follows these steps:

1. Shift the partial remainder and quotient.
2. Subtract the divisor from the partial remainder.
3. Check whether the remainder is negative.
4. If negative, restore the remainder by adding the divisor.
5. Set the appropriate quotient bit.
6. Repeat for all required bits.

The repeated **subtract-and-restore** operations are the main performance bottleneck.

Optimized Solution – Non-Restoring Division

The **non-restoring division algorithm** avoids immediately restoring a negative remainder. Instead:

- If the partial remainder is **positive or zero**, subtract the divisor.
- If the partial remainder is **negative**, add the divisor.

Therefore, the next operation is determined by the sign of the current partial remainder

Restoring Division:

Subtract → Negative? → Restore → Continue

Non-Restoring Division:

Subtract/Add → Check Sign → Continue

Feature	Restoring Division	Non-Restoring Division
Negative remainder	Restore immediately	No immediate restoration
Extra addition	Required	Reduced
Operations	More	Fewer
Execution time	Higher	Lower
Efficiency	Lower	Higher

Testing and Validation

Test Case	Dividend	Divisor	Expected Quotient	Expected Remainder
Test 1	10	2	5	0
Test 2	15	4	3	3
Test 3	20	5	4	0
Test 4	7	3	2	1
Test 5	0	5	0	0

Result

The performance bottleneck in restoring division was identified as the **repeated restoration of negative partial remainders**. Replacing it with the **non-restoring division algorithm** eliminates unnecessary restoration operations, reducing arithmetic operations and improving execution efficiency

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

ANSWER :

Problem Analysis

IEEE 754 single-precision representation uses:

- **1 sign bit**
- **8 exponent bits**
- **23 fraction bits**

The problem occurs when a **very small number is added to a very large number**. The main issues are **exponent alignment, normalization and rounding errors**, as stated in the assessment question.

Identification of the Error

Before floating-point numbers can be added, their exponents must be made equal.

For example:

The exponent difference is very large. During exponent alignment, the significand of the smaller number is shifted to the right by a large number of positions.

As a result, some or all of its significant bits may be lost.

Therefore, the smaller number may have little or no effect on the final result.

Debugging Analysis

IEEE 754 addition follows these steps:

1. Compare the exponents.
2. Align the exponents by shifting the smaller significand.
3. Add the significands.
4. Normalize the result.
5. Round the result.

The major sources of error are:

- **Exponent alignment:** significant bits of the smaller number can be lost.
- **Normalization:** shifting the result may remove significant information.
- **Rounding:** the 23-bit fraction limits the precision of the final result.

Therefore, the root cause is the **limited precision of single-precision representation when numbers have very different magnitudes**.

Corrected Solution

Use **double-precision floating-point representation** for calculations requiring higher accuracy.

Feature	Single Precision	Double Precision
Total bits	32	64
Sign bits	1	1
Exponent bits	8	11
Fraction bits	23	52

Optimization Techniques

The accuracy can be improved by:

- Using double precision for calculations requiring higher accuracy.
- Avoiding unnecessary conversion between different precision formats.
- Reordering calculations to reduce loss of significance.
- Using suitable numerical algorithms when numbers have significantly different magnitudes.
- Applying appropriate rounding techniques.

Testing and Validation

Test Case	Input Condition	Expected Behaviour
Test 1	Similar magnitude numbers	Accurate result
Test 2	Very large + very small	Small value may be lost
Test 3	Very small + very small	Correct normalized result
Test 4	Positive + negative values	Correct sign and magnitude
Test 5	Values requiring rounding	Correct rounded result

Result

The inaccurate result was traced to **exponent alignment, loss of significant bits, normalization and rounding limitations** in single-precision arithmetic. Using **double precision** and appropriate algorithmic adjustments increases the number of significant bits retained and improves the accuracy of floating-point calculations.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)